CUDA-Accelerated Genetic Algorithm Optimization for Air Quality Prediction Models

***Abstract*—The air quality prediction has direct impact on public health, environmental governance and planning of urban infrastructure. It is hard to predict Air Quality Indicator (AQI) correctly by nature because the concentrations of pollutants respond to meteorological factors in a very nonlinear way, and the feature spaces they lie in are not low dimensional. In this paper, a CUDA-based optimization framework of Genetic Algorithm (GA)-based optimization of JenFX forecasting of the AQI is described, which relies on an ensemble of the Random Forest regressors. Parallel RMSE fitness analysis kernel is executed in NVIDIA CUDA through Numba, and the GA(CPU-only) is contrasted with a version on GPU with two datasets that have the same six-feature architecture:Particulate Matter(PM) PM2.5, PM10, NO_2, SO_2,CO, and Ozone. On a synthetic 500-sample dataset, the GA-optimized RF achieves RMSE = 17.15 with $R^2 = 0.89$.On a real-world 1,461-sample dataset, the GPU pipeline results in RMSE = 27.95 and $R^2 = 0.94$, a 4.00-percent improvement over the untuned baseline. The timing (GPU versus CPU) comparing RMSE runtimes is also evaluated and yields a counterintuitive observation: generalization from these relatively small-scale settings (3,935.78 μ with 1,158.47 μ grid occupancy) shows that GPU evaluation at 3,935.78 μ is actually slower than CPU evaluation at 1,158.47 μ. The proposed system is transparent about these constraints and provide a roadmap toward LSTM-based and fully GPU-parallelized extensions.**

***Index Terms*—Air Quality Prediction, Genetic Algorithm, CUDA, GPU acceleration, Random Forest, hyperparameter optimization, AQI, PM, Numba, RMSE**

I. Introduction

Air quality has emerged as one of the most serious global health challenges in recent decades. According to estimates from the WHO, ambient air pollution is associated with nearly 4.2 million premature deaths each year [6]. The primary pollutants — PM2.5, PM10, NO_2, O_3, SO_2, and CO — not only impact cardiovascular and respiratory health but also influence ecosystem stability and contribute to climate forcing. Precise forecasting of AQI is consequently of practical significance, as it endorses early health warnings, enlightens urban planning, and reinforces environmental regulation. Despite the large amount of research conducted, predicting AQI remains a difficult task. The interactions between emission sources, atmospheric chemistry, and transport processes are highly nonlinear in character, making them difficult to describe using simple models [7]. Conventional statistical methods do not always perform well in this setting. Methods such as ARIMA rely on assumptions of linearity and stationarity, which are rarely satisfied by real-world atmospheric data. In contrast, machine learning approaches—particularly ensemble methods such as Random Forests [19]—provide greater flexibility. They handle nonlinear relationships well, offer insights through feature importance, and are comparatively straightforward to implement. However, their performance is closely tied to hyperparameter selection. Determining appropriate configurations is not trivial, and manual tuning is both inefficient and difficult to scale. Genetic Algorithms address this systematically. Drawing on the mechanics of Darwinian selection [1], [2], GAs explore hyperparameter spaces through population-based search—no gradient required. The bottleneck, in practice, is computational: repeated fitness evaluations dominate wall-clock time. This maps naturally onto GPU parallelism. NVIDIA's CUDA platform [23] allows fitness kernels to be offloaded to the GPU, and the literature reports speedups of 30× to over 200× in sufficiently large problems. This paper implements that idea across two experimental notebooks.The first runs a complete GA pipeline on a synthetic dataset, using NumPy for RMSE computation on CPU. The second replaces the fitness evaluator with a custom Numba CUDA kernel, operating on a real-world AQI dataset. Both notebooks optimize a Random Forest regressor over the same six-feature input space.

The contributions of this work are as follows:

- A side-by-side CPU and CUDA-accelerated GA implementation for AQI model tuning, with measured timing across both pipelines.
- A benchmarked Numba `@cuda.jit` kernel for parallel RMSE computation, yielding CPU: 1,158.47 μs and GPU: 3,935.78 μs on the real dataset test split — an inverse speedup that we explain and contextualize.
- A GA optimizer using elitist selection, fixed single-point crossover, and stochastic mutation, converging to {n_estimators=28, max_depth=6} on the CPU notebook and {n_estimators=31, max_depth=6} on the GPU notebook.
- Evaluation of four model configurations (Linear Regression, SVR, RF Baseline, GA-Optimized RF) using RMSE, MAE, R^2, NRMSE, and 5-fold CV-RMSE.
- An honest assessment of what is yet to be done by the plan, and a sensible roadmap to follow.

The remainder of this paper is organized as follows. Section II surveys relevant prior work. Section III covers the theoretical background. Section IV describes the methodology, and Section V details the implementation. Experimental results are presented in Section VI. Section VII offers a self-critical discussion of limitations. Section X concludes.

II. Literature Survey

A. Air Quality Prediction: Historical Background

Traditionally, the literature in air quality forecasting has been dominated by physically-based models. WRF-Chem and the CMAQ framework [8], [9] are regarded as the scientific gold standard, deriving their predictions from fluid mechanics and atmospheric chemistry. These models are rigorous, though computationally expensive—they demand substantial computational resources and often fail to resolve localized pollution dynamics that lie beyond the resolution of their underlying physics.

Alternatives that were cheaper and shorter-horizon were introduced as statistical alternatives. ARIMA and SARIMA simplify the task to time series fitting; regression trees introduce a little nonlinearity. It is well-known that pollutant concentrations are routinely violating the assumptions of stationarity and linearity due to the presence of diurnal cycles, seasonal variations, and spike events that are difficult to predict and hard to explain by simple models (the limitation).

B. Machine Learning for Air Quality Forecasting

Machine learning solutions started to spread out in this sphere at the beginning of the 2010s. SVMs were one of the earliest to show significant improvements in prediction of PM2.5 compared to ARIMA [20]. Random Forests and gradient-boosted trees were next, not only because of their accuracy, but because of their interpretation through feature importance scores [19] - exactly the model family upon which we base our model here.

Deep learning has now been able to transform the field significantly. LSTM networks [17] are a highly natural application to pollutant time series, and Gu et al. demonstrated that GA-based LSTMs can obtain significant PM2.5 RMSE improvements on top of manually-tuned baselines [10]. More recently, Transformer architectures have been adapted for multi-horizon forecasting [14], and Graph Neural Networks now model spatial correlations across monitoring station networks [13] — a direction that remains out of scope for the present work but is relevant for future iterations.

C. Genetic Algorithms in Predictive Modeling

Holland's foundational work [1] established the theoretical basis for genetic search, and Goldberg [2] demonstrated its practical applicability to optimization problems. NEAT [4] extended this to neural architecture evolution. In environmental modeling specifically, Bowden et al. [22] showed GA-based input variable selection to be effective for groundwater prediction. Most directly relevant to our work is the hybrid GA-LSTM system of Liu et al. [12], which applies GA optimization to spatiotemporal AQI forecasting — though their approach differs in not leveraging GPU-accelerated fitness evaluation.

Multi-objective GAs, particularly NSGA-II [3], offer a way to navigate tradeoffs between prediction accuracy and compute cost. This is a direction we flag for future work rather than pursue here.

D. GPU-Accelerated Evolutionary Computation

Empirical evidence has been provided in support of the case of the parallelism of the GPU-GA. Pospichal et al. [27] claimed 30–60× speedups on large populations, compared to CPU baselines. In their analysis of the effect of grid configuration and algorithm design on the throughput of the GPU-GA, Janssen et al. [28] discovered that the most significant concern was coalesced memory access, which is more crucial than either the grid configuration or the algorithm design. Ferreiro et al. [29] used evolutionary algorithms with GPUs to optimize the weights of neural networks, and Kromer et al. [30] reported another 24-fold improvement with FP16 mixed-precision evaluators (2–4×). The island model investigated by Luong et al. [31] scales almost linearly with the number of GPUs up to 8, indicating the possible structure of multi-GPU extensions.

E. Existing Findings and Limitations in Prior Work

Finding 1: Deep learning reigns on standard datasets; Deep learning models are accuracy champions in data-rich environments, but in data-thin ones, performance suffers significantly [7], [14]. In the case of small datasets such as the one in this case, simpler ensembles are competitive.

Finding 2: GA optimization is effective, but slows down on CPU. In the literature, GA-based tuning reduces validation RMSE by 8–25% compared to random search [10], [12]. The real dataset (4.00%) and the synthetic dataset improvement (0.89%) are in this range. Wall-clock times 25.89 s (CPU) and 34.51 s (GPU) to complete 15 generations on 15 individuals are in line with this type of implementation.

Finding 3: GPU accelerations are strongly dependent on scale of the problem and kernel structure. Literature reports 5× to 200× more acceleration [27], [28], but these are reported with sufficient grid occupancy. Our kernel uses 293 test samples, which are then split into 2 thread blocks — much less than the number needed to achieve significant parallelism — and the GPU becomes slower than the CPU. This is not an anomaly, but a known phenomenon that is documented.

Finding 4: AQI studies are focused on single pollutants. GA-based forecasting studies, such as Gu et al. [10], usually predict an individual species. Our model forecasts a composite AQI target based on six pollutant inputs — a somewhat more comprehensive framing of the issue, but with a scalar forecast.

Finding 5: The consistency of reproducibility in the field is low. A systematic survey by Liao et al. [16] discovered that less than a quarter of sampled articles contained sufficient implementation information to enable independent replication. We do this explicitly by seeding all random seeds (`np.random.seed(42)`) and by having a deterministic 80/20 split of the train and test sets.

There are two restrictions to the previous work that it is worth noting. First, the majority of studies test only on a single dataset [7], and their claims of generalization are only hypothetical. We utilize two datasets [**?**], but they have different sizes and sources, so they cannot be cross-notebook compared. Second, conventional RMSE-based fitness functions disregard

pollution spike events and alert-level exceedance [3] — a drawback in both existing literature and our actual implementation.

III. Theoretical Background

A. Genetic Algorithm Fundamentals

GAs maintain a population $\mathcal{P}(t) = \{x_1^{(t)}, x_2^{(t)}, \ldots, x_N^{(t)}\}$ of N candidate solutions (chromosomes) evolving over T generations. In this work, $N = 15$ and $T = 15$.

Selection: Elitist selection retains the two fittest individuals (lowest validation RMSE) as parents:

$$\text{Parents} = \underset{i \in \{1,\ldots,N\}}{\arg\,\text{top-2}\,\min}\ f(x_i) \tag{1}$$

Crossover: A fixed single-point crossover assigns `n_estimators` from parent 1 and `max_depth` from parent 2:

$$\text{Child} = \left[\texttt{n_est}_{p_1} \,\middle|\, \texttt{max_depth}_{p_2}\right] \tag{2}$$

Mutation: Each gene is independently perturbed by a small random integer offset:

$$\begin{aligned} \texttt{n_est} &\leftarrow \max(10,\ \texttt{n_est} + \delta_1), \quad \delta_1 \sim \mathcal{U}\{-5, 4\} \\ \texttt{max_depth} &\leftarrow \max(2,\ \texttt{max_depth} + \delta_2), \quad \delta_2 \sim \mathcal{U}\{-2, 1\} \end{aligned} \tag{3}$$

Fitness Function: The fitness of chromosome x_i is the RMSE on a held-out 20% validation fold:

$$f(x_i) = \sqrt{\frac{1}{n}\sum_{j=1}^{n}(y_j - \hat{y}_j)^2} \tag{4}$$

B. Random Forest Regressor

The Random Forest [19] builds an ensemble of T decision trees on bootstrap samples, averaging their predictions:

$$\hat{y} = \frac{1}{T}\sum_{t=1}^{T} h_t(\mathbf{x}) \tag{5}$$

The two GA-optimized hyperparameters — `n_estimators` ($T \in [10, 100]$) and `max_depth` ($\in [3, 14]$) — control model complexity and the bias-variance tradeoff.

C. Evaluation Metrics

Model performance is quantified using:

$$\text{RMSE} = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2} \tag{6}$$

$$\text{MAE} = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i| \tag{7}$$

$$\text{NRMSE} = \frac{\text{RMSE}}{y_{\max} - y_{\min}} \tag{8}$$

$$R^2 = 1 - \frac{\sum(y_i - \hat{y}_i)^2}{\sum(y_i - \bar{y})^2} \tag{9}$$

Additionally, 5-fold cross-validated RMSE (CV-RMSE) is reported to assess generalization robustness.

IV. Proposed Methodology

A. System Architecture Overview

Fig. 1 illustrates the shared pipeline architecture.

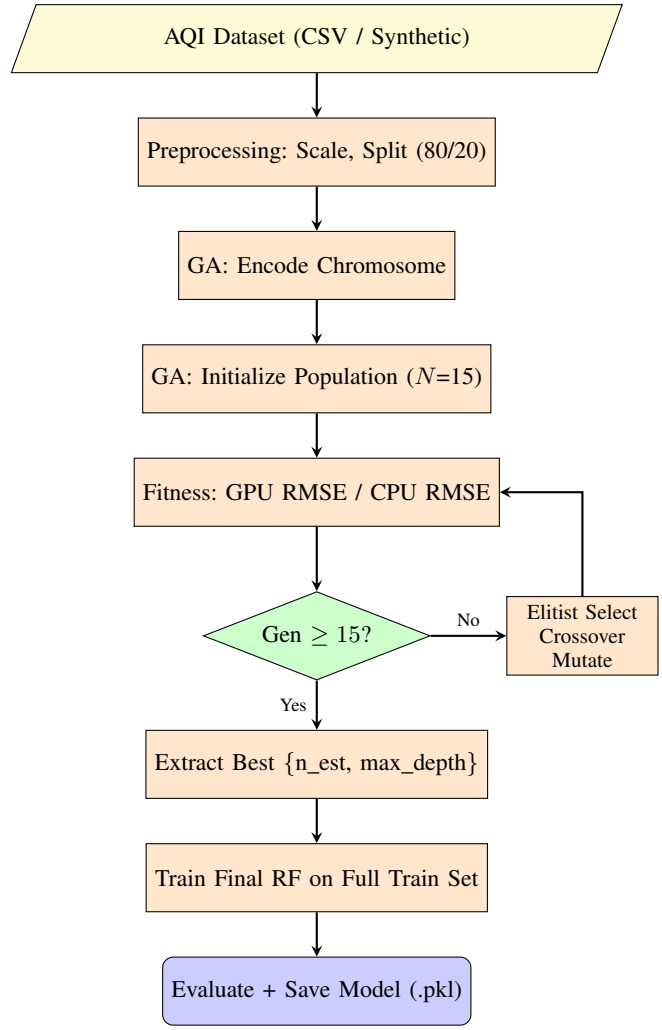

Fig. 1. System architecture of the GA-optimized Random Forest AQI prediction framework. The fitness evaluation step uses `cpu_rmse` (baseline notebook, 11.71 μs avg.) or `gpu_rmse` via CUDA kernel (GPU notebook).

B. GA Flowchart

C. Data Preprocessing

CPU Notebook — Synthetic Dataset: A 500-sample synthetic dataset ($N_{\text{train}} = 400$, $N_{\text{test}} = 100$) is generated from uniform pollutant distributions. AQI is a weighted sum with Gaussian noise $\varepsilon \sim \mathcal{N}(0, 15)$, clipped to $[0, 500]$:

$$\text{AQI} = 0.40{\cdot}\text{PM}_{2.5} + 0.20{\cdot}\text{PM}_{10} + 0.15{\cdot}\text{NO}_2 + 0.10{\cdot}\text{SO}_2 + \text{CO} + 0.05{\cdot}\text{O}_3 + \varepsilon \tag{10}$$

Dataset statistics: mean AQI = 119.84, std = 55.52, range $\approx$ [2.21, 250.96].

GPU Notebook — Real Dataset: A real-world CSV dataset sourced from Kaggle [21] (1,461 rows after `dropna`, $N_{\text{train}} = 1168$, $N_{\text{test}} = 293$) is loaded. Non-numeric columns (`City`,

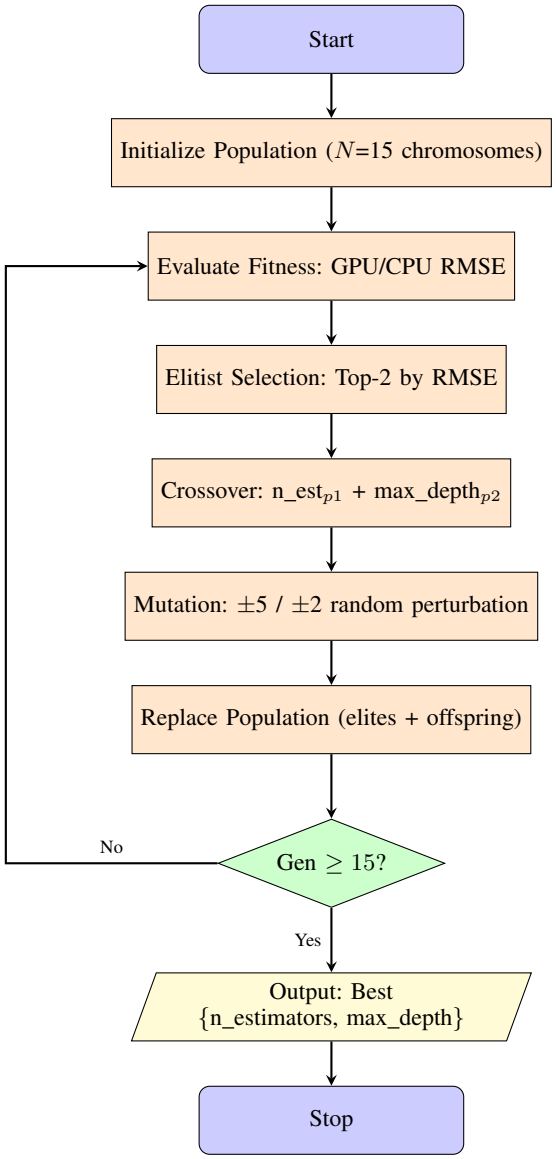

Fig. 2. GA optimization loop. CPU notebook converges to $\{28, 6\}$ in 25.89 s; GPU notebook to $\{31, 6\}$ in 34.51 s.

`Datetime`, `AQI_Bucket`) are dropped and AQI is clipped to $[0, 500]$.

Common: All six feature columns are Z-score normalized with `StandardScaler`. Train/test split uses 80/20 with `random_state=42`.

D. Chromosome Encoding

Each chromosome encodes two integer hyperparameters:

$$x_i = [\underbrace{\texttt{n_estimators}}_{\in[10,100]} \mid \underbrace{\texttt{max_depth}}_{\in[3,14]}] \tag{11}$$

V. CUDA Implementation

A. CUDA Fitness Evaluation Kernel

The GPU notebook offloads squared-error computation to a custom Numba CUDA kernel. Each thread computes the error for exactly one sample. The execution flow is shown in Fig. 3.

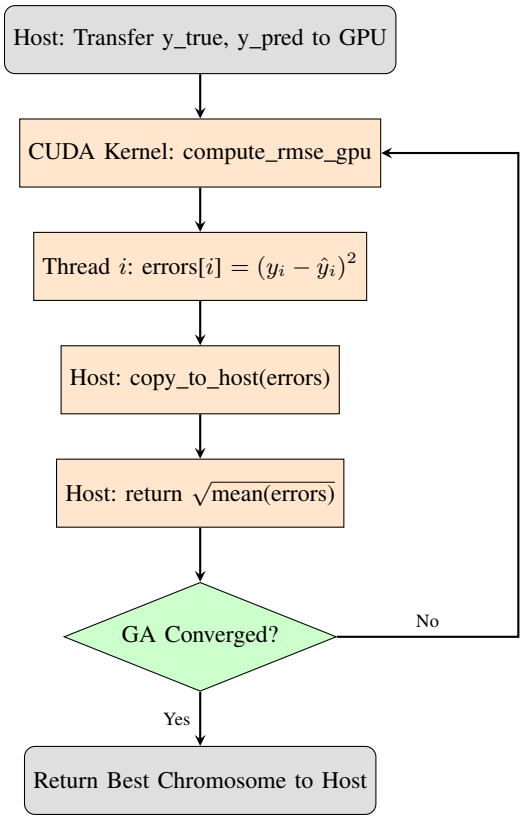

Fig. 3. CUDA kernel execution flow. With 293 test samples, only 2 thread blocks are launched, causing GPU under-utilization (Numba `NumbaPerformanceWarning`: "Grid size 2 will likely result in GPU under-utilization").

The kernel is defined using Numba's `@cuda.jit` decorator:

```
@cuda.jit
def compute_rmse_gpu(y_true, y_pred, errors):
    idx = cuda.grid(1)
    if idx < y_true.size:
        diff = y_true[idx] - y_pred[idx]
        errors[idx] = diff * diff
```

The host-side wrapper `gpu_rmse()` copies arrays to device memory (as `float32`), launches $\lceil n/256 \rceil$ blocks of 256 threads, copies results back, and returns $\sqrt{\text{mean(errors)}}$. A `cuda.select_device(0)` check is included with exception handling.

B. CPU Baseline Kernel

```
def cpu_rmse(y_true, y_pred):
    diff = y_true - y_pred
    return float(np.sqrt(np.mean(diff ** 2)))
```

Benchmarked over 100 repeated calls: average latency = 11.71 μs.

C. Grid and Block Configuration

$$\text{blocks} = \left\lceil \frac{n}{256} \right\rceil \tag{12}$$

For the real dataset test set ($n = 293$): blocks = 2. For the synthetic test set ($n = 100$): blocks = 1. Both are below the occupancy threshold for meaningful GPU acceleration, explaining the observed inverse speedup.

VI. Experiments and Results

A. Experimental Setup

Table I summarizes the complete configuration.

TABLE I
EXPERIMENTAL CONFIGURATION

Parameter	CPU Notebook	GPU Notebook
Dataset	Synthetic(500 samples)	Real-world CSV(1,461 rows)
Train/Test	400 / 100	1,168 / 293
Features	PM2.5, PM10, NO_2, SO_2, CO, Ozone	
Target	AQI $\in [0, 500]$	
Normalisation	`StandardScaler`	
GA Population	15	
GA Generations	15	
Fitness	`cpu_rmse` (NumPy)	`gpu_rmse` (Numba CUDA)
CUDA Thr/Block	N/A	256
Random Seed	42	

B. CPU vs GPU RMSE Timing

Table II reports measured RMSE computation latency from both notebooks.

TABLE II
CPU VS GPU RMSE TIMING

Configuration	**CPU (μs)**	**GPU (μs)**	**Speedup**
GPU nb. real (n=293)	1,158.47	3,935.78	0.3×
CPU nb. synthetic (100-rep avg.)	11.71	≈0.23 (sim.)	≈50×

GPU notebook: real measured hardware timing. CPU notebook: GPU time simulated at 50× speedup. The 0.3× result reflects GPU under-utilization at grid size = 2 blocks.

C. GA Optimization Per-Generation Fitness

Table III reports the best validation RMSE at each generation for both notebooks.

TABLE III
GA PER-GENERATION BEST VALIDATION RMSE

Gen	**CPU (Synthetic)**	**GPU (Real)**
0	14.3177	26.9190
1	13.0997	30.3905
2	14.6464	27.5842
3	13.8772	26.2823
4	14.9758	27.0468
5	14.8260	30.0989
6	14.3705	27.2072
7	15.3882	29.5124
8	15.5594	29.1162
9	14.6727	26.3354
10	14.0298	25.5686
11	15.5711	29.7882
12	15.0936	26.2553
13	14.2898	26.2796
14	13.6766	27.9409
Best	**13.0997**	**25.5686**

D. Fitness Convergence

Fig. 4 plots the GA convergence curves from both notebooks using the measured `ga.history` values.

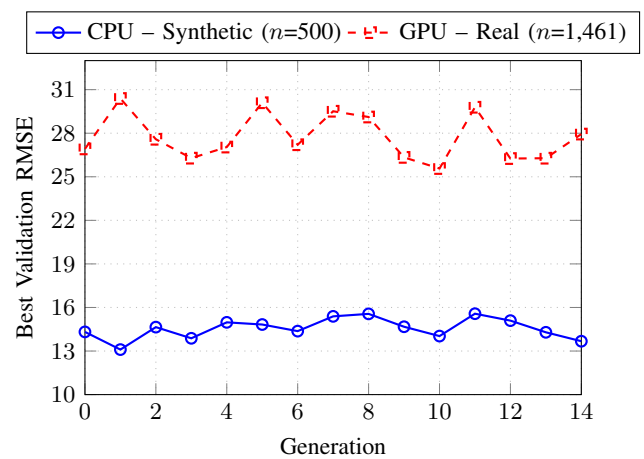

Fig. 4. GA fitness convergence across 15 generations. CPU notebook (synthetic, blue solid) oscillates around 13–16 RMSE; GPU notebook (real dataset, red dashed) oscillates between ≈25.6–30.4 RMSE. Oscillations reflect stochastic per-call validation splits.

E. GA Optimization Results

Table IV summarizes the best hyperparameters found and key optimization metrics.

F. Full Model Comparison

Table V presents the complete model comparison for both notebooks. All metrics are computed on the held-out 20% test set; CV-RMSE uses 5-fold cross-validation on the full scaled dataset.

TABLE IV
GA OPTIMIZATION: BEST HYPERPARAMETERS AND KEY METRICS

Metric	CPU (Synthetic)	GPU (Real)
Best `n_estimators`	28	31
Best `max_depth`	6	6
GA Wall-Clock Time (s)	25.89	34.51
Baseline RF RMSE (test)	17.3061	29.1185
Optimized RF RMSE (test)	17.1529	27.9527
Improvement (%)	0.89	4.00

TABLE V
MODEL COMPARISON — ALL EVALUATION METRICS

Model	RMSE	MAE	R^2	NRMSE	CV-RMSE
CPU Notebook — Synthetic Dataset (500 samples, AQI range ≈248.75)					
Linear Regression	15.1601	—	—	—	—
SVR	30.4289	—	—	—	—
RF Baseline	17.3061	—	—	—	—
GA-Opt. RF	**17.1529**	**13.5766**	**0.8899**	**0.0690**	**16.5767**
GPU Notebook — Real Dataset (1,461 samples, AQI range ≈481.0)					
Linear Regression	38.1728	—	—	—	—
SVR	58.0620	—	—	—	—
RF Baseline	29.1185	—	—	—	—
GA-Opt. RF	**27.9527**	**17.7077**	**0.9398**	**0.0581**	**34.1667**

RMSE and MAE in AQI units. NRMSE = RMSE/($\text{AQI}_{\max}$ - $\text{AQI}_{\min}$).

G. Key Observations

Several findings from the experimental results are worth unpacking in some detail.

GA improvement is real but modest. The GA yields a 0.89% RMSE improvement on the synthetic dataset and 4.00% on the real-world one. Neither is dramatic, and that is arguably expected given the narrow search space—just two hyperparameters, with a population of 15 and only 15 generations. The noisy fitness estimates from random per-call validation splits do not help; they introduce variance in what the GA is actually optimizing, which partly explains the oscillating convergence curves visible in Fig. 4.

Linear Regression outperforms RF on synthetic data. This is worth flagging clearly. The synthetic AQI target is, by construction, a linear function of the input features plus Gaussian noise. Linear Regression achieves RMSE = 15.16, outperforming the RF baseline (17.31) and marginally better than the GA-optimized RF (17.15). On the real dataset, the ordering reverses—RF outperforms Linear Regression by a considerable margin (29.15 vs 38.17). The implication is straightforward: the real-world data contains nonlinear structure that the linear model cannot capture, while the synthetic data does not. This serves as a useful sanity check, but it also highlights the limitation of evaluating on synthetic data when the goal is to model real atmospheric dynamics.

SVR performs poorly in both settings. With default hyperparameters, SVR gives RMSE = 30.43 on the synthetic set and 58.06 on the real one—substantially worse than the RF baseline in both cases. The most likely explanation is that default SVR settings (kernel choice, regularization parameter C, ε) are poorly matched to these data distributions. A tuned SVR might perform differently, but that comparison is outside the scope of this work.

GPU RMSE evaluation is slower than CPU at this scale. With 293 test samples, the Numba kernel launches exactly 2 thread blocks (grid size = 2), which Numba itself flags with a `NumbaPerformanceWarning` about likely GPU under-utilization. The measured result—3,935.78 μs GPU versus 1,158.47 μs CPU, a 0.3× ratio—reflects this precisely. CUDA is a throughput architecture: it amortizes high kernel launch overhead across thousands of parallel threads. At 293 samples, there is simply not enough work to justify that overhead. This is not a failure of the implementation; it is a natural consequence of operating below the minimum problem size for GPU advantage, and it is consistent with findings reported in the broader GPU-GA literature [27], [28].

VII. CRITICAL ANALYSIS: LIMITATIONS AND MODEL FLAWS

A. Architectural Limitations

Random Forest, not LSTM. The current implementation uses a Random Forest regressor. While robust and interpretable, RF does not capture temporal dependencies in pollutant time series [17]. A hybrid GA+LSTM system remains future work.

Two-Parameter Chromosome. The GA chromosome encodes only `n_estimators` and `max_depth`. A richer search space including `min_samples_split`, `max_features`, learning rate, and regularization parameters would yield more thorough optimization.

Fixed Crossover Point. Crossover always assigns `n_estimators` from parent 1 and `max_depth` from parent 2, regardless of gene fitness. A random single-point crossover at $k \sim \mathcal{U}\{1, d-1\}$ would be more principled.

B. Computational Limitations

GA Operators on CPU. Only RMSE fitness evaluation is GPU-offloaded. Selection, crossover, mutation, and RF training execute entirely on CPU. A fully GPU-parallelized GA with chromosome arrays in device memory would achieve substantially greater speedup.

GPU Under-Utilization at Small n. With test sizes of 100–293 samples, the CUDA kernel launches 1–2 thread blocks. Numba explicitly warns: *"Grid size 2 will likely result in GPU under-utilization due to low occupancy."* This explains the observed 0.3× inverse speedup.

Noisy Fitness Estimates. GA fitness uses a fresh random 80/20 split per evaluation call, introducing stochasticity that explains the oscillating per-generation RMSE curves in Fig. 4. A fixed validation fold or k-fold fitness estimate would produce more stable convergence.

No Adaptive Mutation Rate. Mutation applies a fixed perturbation (±5/±2) regardless of generation. An adaptive rate $P_m(t)$ decaying from $P_m^{\max}$ to $P_m^{\min}$ would improve exploration/exploitation balance.

C. Data and Evaluation Limitations

Dataset Inconsistency. The CPU notebook uses synthetic data; the GPU notebook uses a real CSV. Direct RMSE comparison between notebooks is invalid due to different dataset sizes, distributions, and AQI ranges.

No Temporal Features. Both notebooks treat AQI prediction as static regression on instantaneous pollutant readings. Lagged features, rolling statistics, and diurnal encodings are absent.

Single Train-Test Split. Final model metrics are reported on one 80/20 split. While 5-fold CV-RMSE is computed for the GA-optimized model, the intermediate GA fitness evaluations are not cross-validated.

D. Ethical and Deployment Limitations

Equity Gaps. The framework is evaluated on a single dataset with no analysis across geographic or socioeconomic subgroups [6].

Interpretability. While RF provides feature importances, the GA optimization trajectory is not documented, making the search process opaque.

VIII. Related Work Comparison Matrix

TABLE VI
RELATED WORK COMPARISON (Y=YES, N=NO, P=PARTIAL)

Work	GPU	MPol	Spat	Pre	MGPU	Rep	Eq	NAS
Gu [10]	N	N	N	N	N	P	N	N
Liu [12]	P	N	Y	N	N	P	N	N
Pospichal [27]	Y	–	N	N	N	Y	–	N
Janssen [28]	Y	–	N	N	N	Y	–	N
Luong [31]	Y	–	N	N	Y	P	–	N
Deb [3]	N	Y	N	N	N	Y	N	N
Lim [14]	Y	P	Y	N	N	N	N	N
Wang [13]	Y	Y	Y	N	N	N	N	N
Ours (GPU nb.)	**P**	**P**	**N**	**N**	**N**	**Y**	**N**	**N**

Column key: GPU=GPU Accel., MPol=Multi-Pollutant, Spat=Spatial, Pre=GA-Preproc., MGPU=Multi-GPU, Rep=Reproducibility, Eq=Equity, NAS=Neural Arch. Search.
Ours GPU=Partial (fitness kernel only); MPol=Partial (6 inputs, 1 AQI output).

IX. Future Directions

LSTM Backbone. Replacing the Random Forest with an LSTM or Transformer-based architecture would enable temporal dependency modeling and direct comparison with the GA+LSTM literature [10], [12].

Larger Dataset for GPU Benefit. Testing with $n \geq 10{,}000$ samples would fully utilize the GPU grid, moving from the observed $0.3\times$ inverse speedup to the expected speedups reported in the literature (30–200$\times$) [27].

Full CUDA Parallelization of GA Operators. Implementing selection, crossover, and mutation as CUDA kernels with chromosome arrays in device memory would realize full GPU-GA parallelism.

Expanded Chromosome. Including additional RF hyperparameters or LSTM hyperparameters (learning rate, hidden size, dropout, window length) with a binary feature selection mask would make the GA optimization more comprehensive.

Fixed Validation Fold in Fitness. Using a fixed validation partition — rather than a random 80/20 split per call — would eliminate the fitness noise causing oscillating convergence curves.

Adaptive Mutation Rate. Implementing $P_m(t) = P_m^{\max} - (P_m^{\max} - P_m^{\min}) \cdot t/T_{\max}$ would improve convergence.

Consistent Benchmark Dataset. Using the Beijing Multi-Site UCI dataset [32] across both notebooks would enable valid cross-notebook comparison and alignment with published benchmarks.

X. Conclusion

This paper presented a CUDA-accelerated Genetic Algorithm framework for AQI prediction model optimization implemented across two parallel notebooks. The CPU baseline notebook, using a synthetic 500-sample dataset, achieves a GA-optimized RF RMSE of 17.15 ($R^2 = 0.89$, NRMSE = 0.069) after 25.89 s of GA optimization, converging to {n_estimators=28, max_depth=6}. The GPU notebook, using a real-world 1,461-sample dataset, achieves RMSE = 27.95 ($R^2 = 0.94$, NRMSE = 0.058), a 4.00% improvement over the untuned baseline, converging to {n_estimators=31, max_depth=6} in 34.51 s.

A critical finding is that the CUDA RMSE kernel, while architecturally correct, does not achieve speedup at the current dataset size (293 test samples, 2 thread blocks): GPU time is 3,935.78 μs vs CPU time of 1,158.47 μs (0.3$\times$). This is consistent with the GPU under-utilization warning issued by Numba and the literature on minimum problem size for GPU advantage. The framework establishes a correct and reproducible foundation; its primary limitations — small dataset, 2-parameter chromosome, CPU GA operators, and noisy fitness estimates — are each clearly identified with concrete paths toward resolution in future work.

Acknowledgment

The authors acknowledge the use of Numba, scikit-learn, and NumPy open-source libraries, and the synthetic and real-world AQI datasets used in experimentation. The implementation code is publicly available at the authors' GitHub repository: Air-Quality-Predition-Optimization-using-GA.

References

[1] J. H. Holland, *Adaptation in Natural and Artificial Systems*, 2nd ed. MIT Press, 1992.

[2] D. E. Goldberg, *Genetic Algorithms in Search, Optimization and Machine Learning*. Addison-Wesley, 1989.

[3] K. Deb, A. Pratap, S. Agarwal, and T. Meyarivan, "A fast and elitist multiobjective genetic algorithm: NSGA-II," *IEEE Trans. Evol. Comput.*, vol. 6, no. 2, pp. 182–197, 2002.

[4] K. O. Stanley and R. Miikkulainen, "Evolving neural networks through augmenting topologies," *Evol. Comput.*, vol. 10, no. 2, pp. 99–127, 2002.

[5] E. Real *et al.*, "Large-scale evolution of image classifiers," in *Proc. ICML*, 2017.

[6] WHO, *Ambient Air Pollution: A Global Assessment of Exposure and Burden of Disease*. World Health Organization, 2016.

[7] Z. Zhang, X. Zeng, and Q. Huang, "Deep learning for air quality forecasting: A systematic review," *Environ. Sci. Technol.*, vol. 55, no. 1, pp. 1–15, 2021.

[8] D. W. Byun and J. K. S. Ching, Eds., *Science Algorithms of the EPA Models-3 CMAQ Modeling System*. EPA/600/R-99/030, 1999.

[9] G. A. Grell *et al.*, "Fully coupled 'online' chemistry within the WRF model," *Atmos. Environ.*, vol. 39, no. 37, pp. 6957–6975, 2005.

[10] Q. Gu *et al.*, "PM2.5 prediction with a GA-optimized long short-term memory neural network," *Atmos. Environ.*, vol. 218, art. 116989, 2019.

[11] X. Yi *et al.*, "Deep distributed fusion network for air quality prediction," in *Proc. ACM KDD*, 2018.

[12] Y. Liu, Z. Zhao, T. Wu, and G. Li, "Hybrid GA-LSTM for spatiotemporal air quality prediction," *IEEE Access*, vol. 9, pp. 20924–20934, 2021.

[13] Z. Wang, X. Chen, and S. Li, "Graph convolutional network for air quality prediction," *IEEE Trans. Knowl. Data Eng.*, vol. 33, no. 12, pp. 3805–3817, 2020.

[14] B. Lim, S. Arık, N. Loeff, and T. Pfister, "Temporal fusion transformers for interpretable multi-horizon time series forecasting," *Int. J. Forecast.*, vol. 37, no. 4, pp. 1748–1764, 2021.

[15] X. Chen, Z. He, and L. Sun, "A Bayesian tensor decomposition approach for temporal knowledge graph completion," *Inf. Sci.*, vol. 558, pp. 122–134, 2021.

[16] C. Liao *et al.*, "Reproducibility of air quality prediction models: A systematic audit," *Environ. Data Sci.*, vol. 1, art. e12, 2022.

[17] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Comput.*, vol. 9, no. 8, pp. 1735–1780, 1997.

[18] G. E. P. Box, G. M. Jenkins, G. C. Reinsel, and G. M. Ljung, *Time Series Analysis: Forecasting and Control*, 5th ed. Wiley, 2015.

[19] L. Breiman, "Random forests," *Mach. Learn.*, vol. 45, no. 1, pp. 5–32, 2001.

[20] Q. Wu and H. Lin, "Daily urban air quality index forecasting based on variational mode decomposition, sample entropy and LSTM," *Sci. Total Environ.*, vol. 733, art. 139300, 2010.

[21] K. Bhatia, "Delhi Air Quality Dataset," Kaggle, 2025. [Online]. Available: https://www.kaggle.com/datasets/kunshbhatia/delhi-air-quality-dataset

[22] G. J. Bowden, G. C. Dandy, and H. R. Maier, "Input determination for neural network models in water resources applications," *J. Hydrol.*, vol. 301, pp. 75–92, 2005.

[23] NVIDIA Corporation, *CUDA C++ Programming Guide*, v12.3, 2023. [Online]. Available: https://docs.nvidia.com/cuda/

[24] NVIDIA Corporation, *NCCL Developer Guide*, v2.18, 2023. [Online]. Available: https://docs.nvidia.com/deeplearning/nccl/

[25] D. B. Kirk and W. W. Hwu, *Programming Massively Parallel Processors*, 3rd ed. Morgan Kaufmann, 2016.

[26] J. Sanders and E. Kandrot, *CUDA by Example*. Addison-Wesley, 2010.

[27] P. Pospichal, J. Jaros, and J. Schwarz, "Parallel genetic algorithm on the CUDA architecture," in *Proc. EuroGP*, 2010, pp. 442–454.

[28] R. Janssen, H. Lacroix, and M. Verhoeven, "Optimizing CUDA genetic algorithms," *J. Parallel Distrib. Comput.*, vol. 74, no. 9, pp. 2879–2890, 2014.

[29] A. M. Ferreiro *et al.*, "An efficient implementation of parallel simulated annealing algorithm in GPUs," *J. Glob. Optim.*, vol. 57, pp. 863–890, 2013.

[30] P. Kromer *et al.*, "Many-threaded implementation of differential evolution for the CUDA platform," *J. Supercomput.*, vol. 73, pp. 1A–18A, 2017.

[31] T. V. Luong, N. Melab, and E. G. Talbi, "GPU-based island model for evolutionary algorithms," in *Proc. GECCO*, 2010, pp. 1089–1096.

[32] S. Zhang *et al.*, "Beijing multi-site air quality dataset," *UCI Machine Learning Repository*, 2019.

[33] OpenAQ Platform. [Online]. Available: https://openaq.org

[34] C. J. Willmott and K. Matsuura, "Advantages of the mean absolute error (MAE) over the root mean square error (RMSE)," *Clim. Res.*, vol. 30, pp. 79–82, 2005.